Plotly Express Quick Reference

Scatter plots, line plots and histograms | Based on the class notebook Plotly.ipynb

Plotly Express (`px`) builds interactive charts from a pandas DataFrame in just one or two lines of code. This sheet follows the order of your class notes: setup, scatter plot, line plot, saving to HTML, and histogram.

1. Setup and Data

1.1 Imports, loading data and creating a subset

Run this cell first. All later examples use `px`, `student` and `student_marks`.

EXAMPLE

```
import pandas as pd
import plotly.express as px

student = pd.read_csv('student_dataset_complete.csv')

# Subset of columns used later for the histograms
student_marks = student[['Marks', 'study_hours', 'attendance_rate', 'gender']]
```

WHAT EACH LINE DOES

Line	Meaning
`import pandas as pd`	Loads pandas, which reads and works with tables of data. `pd` is its short name.
`import plotly.express as px`	Loads Plotly Express, which draws the charts. `px` is its short name.
`pd.read_csv('...')`	Reads a CSV file into a DataFrame (a table). Here it is stored in `student`.
`student[['Marks', ...]]`	Creates a smaller table that keeps only the listed columns. Here it is stored in `student_marks`.

NOTES AND COMMON MISTAKES

- The CSV file must be in the same folder as your notebook, or you must give the full path. Otherwise Python shows `FileNotFoundError`.
- To select columns use **double square brackets**: the outer pair selects, the inner pair is a list of names. `student['Marks', 'gender']` (one pair) gives a `KeyError`; `student[['Marks', 'gender']]` works.
- If `import plotly.express as px` gives `ModuleNotFoundError`, install it once with `pip install plotly`, then run the cell again.
- Style: write keyword arguments without spaces around `=`, for example `x='time_study'`. Spaces still work, but this is the standard Python style and is used in this sheet.

1.2 The student dataset

`student` has 100 rows and 14 columns: `number_courses`, `time_study`, `Marks`, `student_id`, `class_level`, `subject`, `study_hours`, `test_score`, `week`, `attendance_rate`, `gender`, `hostel`, `Tshirt_size`, `class_name`. These are the columns used in the notebook:

Column	Type	Example values	Used for
`time_study`	number	4.508, 0.096	Scatter plot (x)
`Marks`	number	19.202, 7.734	Scatter plot (y)
`gender`	text	F, M	Color (scatter plot and histogram)
`Tshirt_size`	whole number	1, 2, 3, 4	Point size (scatter plot)
`hostel`	0 or 1	0, 1	Extra hover information (scatter plot)
`attendance_rate`	number	0.89, 0.98	Histogram (x)
`study_hours`	number	4.508, 0.096	Kept in `student_marks`, not plotted

- In the rows shown in your notebook, `time_study` and `study_hours` hold the same values, and so do `Marks` and `test_score`.

1.3 Viewing a DataFrame

Type the DataFrame name on the last line of a cell to display it. Jupyter shows the first 5 and last 5 rows, with `...` for the rows in between.

EXAMPLE

```
student_marks
```

OUTPUT

```
     Marks  study_hours  attendance_rate gender
0   19.202        4.508             0.89      F
1    7.734        0.096             0.98      M
2   13.811        3.133             0.83      F
3   53.018        7.909             0.92      F
4   55.299        7.811             0.80      M
..     ...          ...              ...    ...
95  19.128        3.561             0.94      F
96   5.609        0.301             0.83      M
97  41.444        7.163             0.97      M
98  12.027        0.309             0.98      F
99  32.357        6.335             0.95      M

[100 rows x 4 columns]
```

- Typing `student` the same way shows all 14 columns. Only the **last** expression in a cell is displayed.

1.4 The basic workflow for every chart

1. Create the figure with a `px` function and store it in a variable, usually `fig`.
2. Display it with `fig.show()`. If you forget this line, no chart appears.

- Charts are interactive: hover over a chart to see the values. `fig` is only a variable name, and each new `fig = px...` line replaces the previous chart.

2. Scatter Plot: px.scatter()

A scatter plot shows **one dot per row** and is used to see the relationship between two numeric columns. Here we ask: do students who study longer get higher marks? (This topic was marked as optional in your notes.)

SYNTAX (REPLACE THE NAMES WITH YOUR OWN)

```
fig = px.scatter(df, x='col_x', y='col_y', color='col_c', size='col_s',
hover_data=['col_h'])
```

PARAMETERS

Parameter	What it does	In the example
`x`	Column for the horizontal axis	`'time_study'`
`y`	Column for the vertical axis	`'Marks'`
`color`	Gives each group its own color and adds a legend	`'gender'`
`size`	Sets the dot size from a numeric column	`'Tshirt_size'`
`hover_data`	Extra columns shown when you hover over a dot (a list)	`['hostel']`

EXAMPLE

```
fig = px.scatter(student, x='time_study', y='Marks',
                 color='gender',
                 size='Tshirt_size',
                 hover_data=['hostel'])

fig.show()
```

OUTPUT

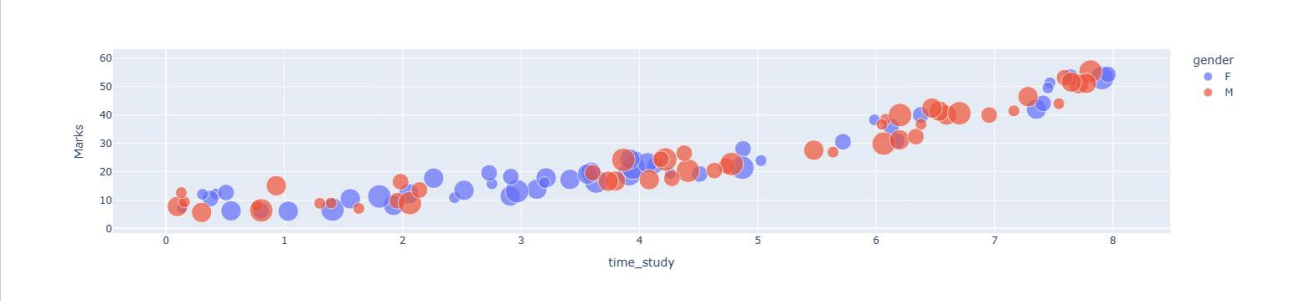


Figure 1. Scatter plot of Marks against time_study, colored by gender, sized by Tshirt_size.

How to read it: the dots rise from left to right, so students who study more tend to get higher marks. The two colors show gender, and bigger dots mean a larger `Tshirt_size`.

NOTES AND COMMON MISTAKES

- Column names go in quotes and must match exactly, including capital letters. `y='marks'` gives a `ValueError` because the column is `Marks`. Without quotes you get a `NameError`.
- `size` needs a **numeric** column. A text column such as `'gender'` gives a `TypeError`. `Tshirt_size` works because it is stored as numbers.
- `hover_data` takes a list, so use square brackets. Add more columns inside the list, for example `['hostel', 'week']`.

3. Line Plot: px.line()

A line plot joins the points in order from left to right. It is used to show **how a value changes over time**. Here we look at course enrollment for each year.

SYNTAX (REPLACE THE NAMES WITH YOUR OWN)

```
fig = px.line(df, x='col_x', y='col_y', markers=True)
```

PARAMETERS

Parameter	What it does	In the examples
`x`	Column for the horizontal axis (here, the year)	`'Year'`
`y`	Column for the vertical axis (the value to track)	`'Programming'` (3.1), `'Digital Marketing'` (3.2)
`markers`	Optional. `True` adds a dot on every data point. Without it, only the line is drawn.	`markers=True`

3.1 Basic line plot

EXAMPLE

```
enrollment = pd.read_csv('enrollment_data.csv')

fig = px.line(enrollment, x='Year', y='Programming')

fig.show()
```

DATA BEHIND THE CHART

Year	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024
Programming	589	862	1085	1150	1439	1715	1785	1995	2236	2442
Digital Marketing	734	963	1157	1404	1497	1579	1948	2073	2310	2338

OUTPUT

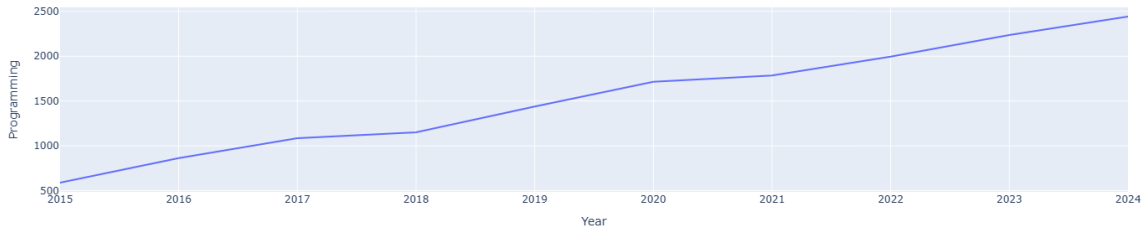


Figure 2. Line plot of Programming enrollment by Year.

How to read it: Programming enrollment rises every year, from 589 in 2015 to 2442 in 2024.

3.2 Line plot with markers

EXAMPLE

```
fig = px.line(enrollment, x='Year', y='Digital Marketing', markers=True)

fig.show()
```

OUTPUT

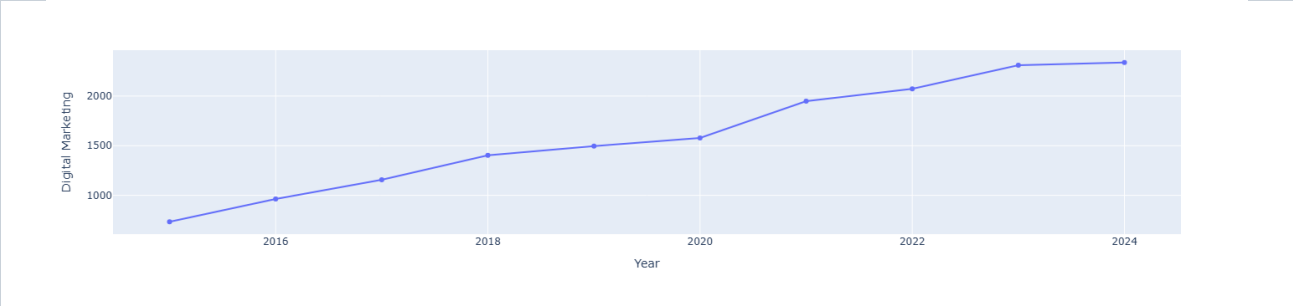


Figure 3. Line plot of Digital Marketing enrollment by Year, with markers.

NOTES AND COMMON MISTAKES

- Write `True` with a capital **T** and **no quotes**. `markers=true` gives a `NameError`. With quotes, even `markers='False'` still turns markers on, because any non-empty text counts as true.
- A column name with a space, such as `'Digital Marketing'`, is fine inside quotes. The spelling and spacing must match the CSV exactly.
- The data comes from `enrollment`, a different DataFrame from `student`. Use the DataFrame that contains the columns you name.

4. Saving a Plot as an HTML File: fig.write_html()

`write_html()` saves a figure as a web page that you can open in any browser. The chart stays interactive, so you can share it with other people.

SYNTAX

```
fig.write_html('file_name.html')
```

EXAMPLE

```
# Create an HTML file
fig.write_html('digital_marketing_data.html')
```

Nothing is printed in the notebook. The file `digital_marketing_data.html` appears in the same folder as your notebook. Double-click it to open it in your browser.

NOTES AND COMMON MISTAKES

- It saves whichever chart `fig` holds **at that moment**. In class, `fig` was the Digital Marketing line plot with markers (section 3.2).
- The file name must end with `.html`. Saving again with the same name overwrites the old file.
- The file is several MB because it contains the Plotly library, so it opens offline.

5. Histogram: px.histogram()

A histogram splits the values of **one numeric column** into ranges called **bins** and counts how many rows fall in each bin. It shows how the values are spread out. Plotly counts the rows for you and labels the y-axis `count`.

SYNTAX (REPLACE THE NAMES WITH YOUR OWN)

```
fig = px.histogram(df, x='col_x', color='col_c', nbins=5)
```

PARAMETERS

Parameter	What it does	In the examples
`x`	The numeric column to count	`'attendance_rate'`
`color`	Optional. Draws separate bars for each group and adds a legend	`'gender'`
`nbins`	Optional. The number of bins (bars) you would like	`5`

5.1 Basic histogram

EXAMPLE

```
fig = px.histogram(student_marks, x='attendance_rate')

fig.show()
```

OUTPUT

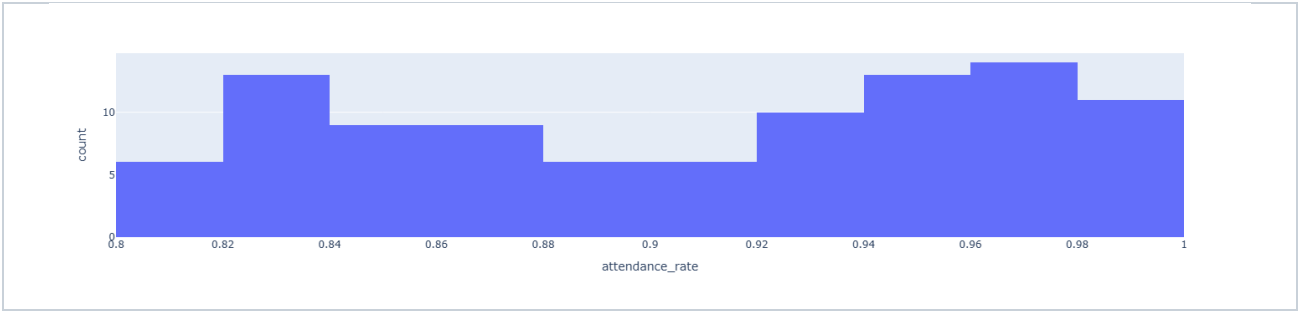


Figure 4. Histogram of attendance_rate for all 100 students.

How to read it: each bar is one attendance-rate range. A taller bar means more students in that range. The tallest bar here is near 0.96 to 0.98.

5.2 Histogram with color and nbins

EXAMPLE

```
fig = px.histogram(student_marks, x='attendance_rate', color='gender', nbins=5)

fig.show()
```

OUTPUT

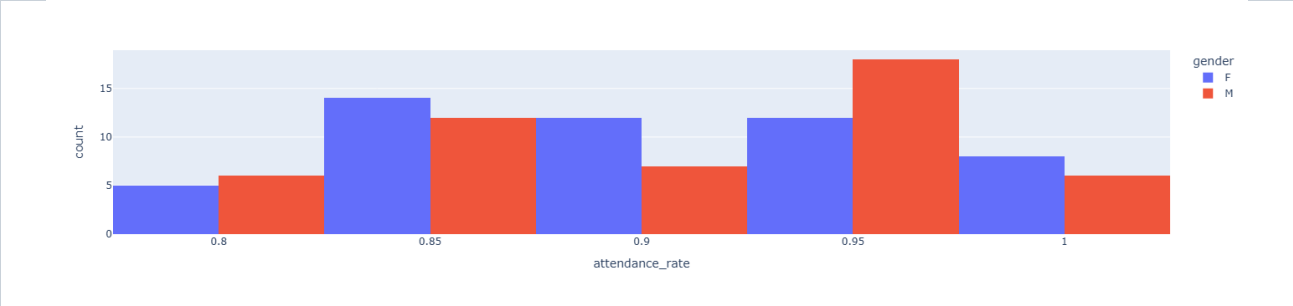


Figure 5. Histogram of attendance_rate split by gender, with about 5 bins.

NOTES AND COMMON MISTAKES

- `color='gender'` draws separate bars for F and M. A legend explains the colors.
- `nbins=5` asks for about 5 bars. Plotly treats it as the **maximum** number of bins wanted and picks neat bin edges, so the exact number can differ. Fewer bins give a simpler picture; more bins give more detail.
- In your class output the F and M bars sit side by side. In some Plotly versions they are stacked instead. Both are normal.
- You only need `x`. Plotly counts the rows for you, so no `y` column is needed.
- Use only columns that exist in the DataFrame you pass in. `student_marks` has just 4 columns, so `x='hostel'` gives a `ValueError`. Use `student` for that column.

6. Quick Revision Summary

6.1 Chart types

Chart	Function	Use it to show	Parameters used in class
Scatter	`px.scatter()`	The relationship between two numeric columns	`x`, `y`, `color`, `size`, `hover_data`
Line	`px.line()`	How a value changes over time	`x`, `y`, `markers`
Histogram	`px.histogram()`	How the values of one column are spread out	`x`, `color`, `nbins`

6.2 Other commands

Task	Command
Import pandas and Plotly Express	`import pandas as pd` and `import plotly.express as px`
Read a CSV file	`df = pd.read_csv('file.csv')`
Keep only some columns	`df_small = df[['col1', 'col2']]`
Show a chart	`fig.show()`
Save a chart as an HTML file	`fig.write_html('file_name.html')`